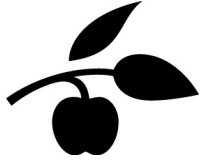


# Model-Driven Software Development

## Types

**SDU** 

Previous class

Programming Experience

# Summary

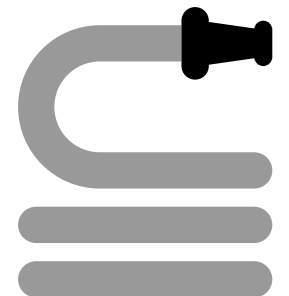
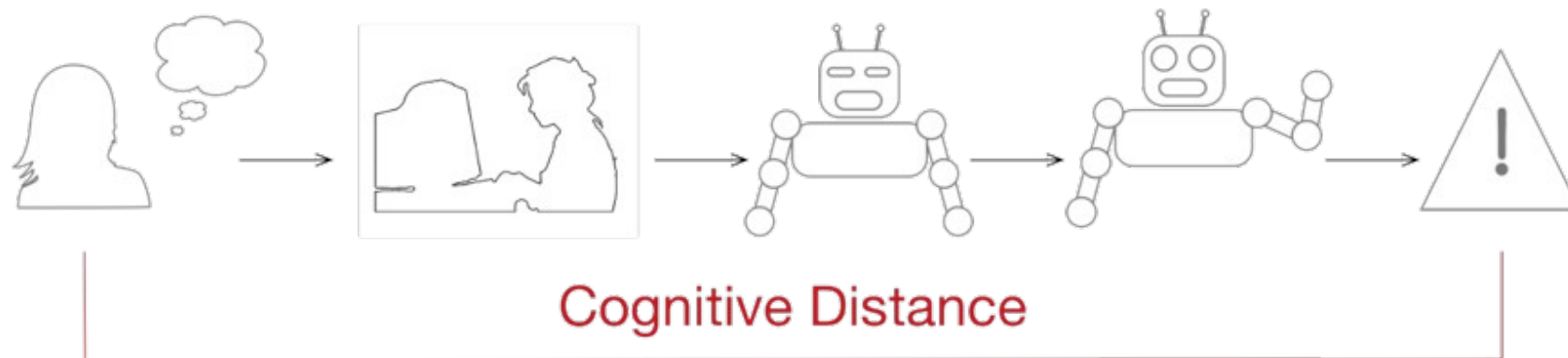
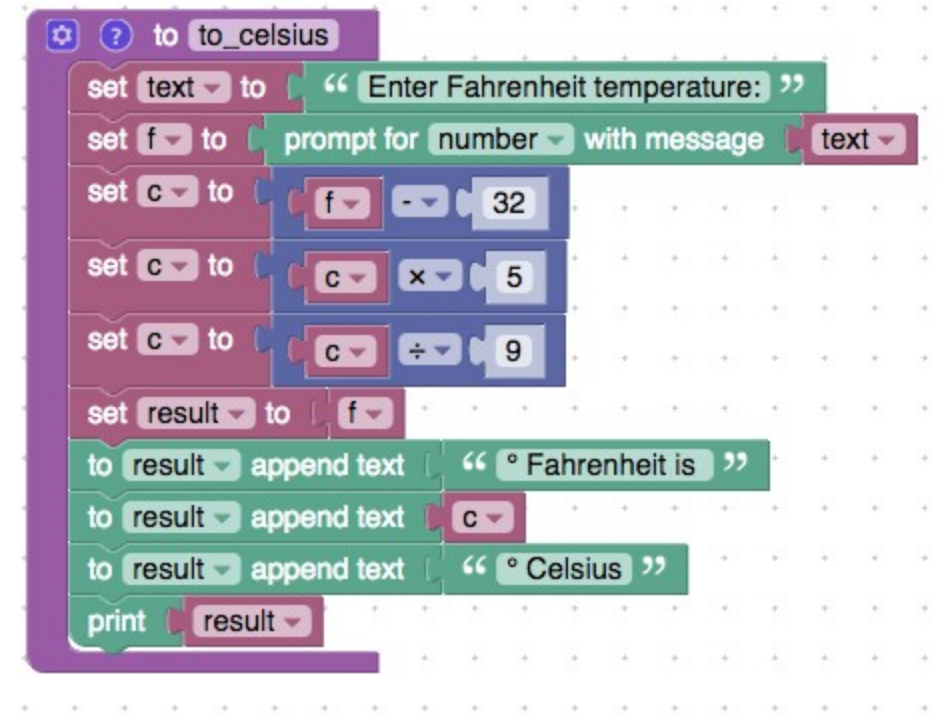
Programming eXperience (PX)

Live Programming

How to: Traceability

Visual DSLs

Next Class → Types



# What aspect do we want to improve?

Improving:

Usability

Learnability

Comprehension

Efficiency of coding

Enjoyment

...

For whom?

Coders / Developers

Designers

Architects

Engineers

Machine Operators

Artists

Citizens (low-code)

...

Development practices

Coding

Debugging

Refactoring

Design

Program Comprehension

...

# Meaningful Feedback

Rule of thumb: Mental Models / Metamodel

State machine → States and Transitions

Interactive Fiction → Graph

Class Description → Class Diagram

...

However, it may not be enough

Mental model may not be clear enough → Interactive Fiction, Graph, and Player

Metamodel heavily dependent on domain (it is context specific)

What about feedback from programs in GPL?

There may be more information developers need to understand execution

# Live Programming

Combination of 2 concepts

Immediate Connection

Changes are reflected immediately

Meaningful Feedback

The program gives proper feedback to developers

*Logs are not proper feedback*

# Level of Liveness

Level 1: Informative

Visual representation

Level 2: Informative and significant

Executable visual representation

Level 3: Informative, significant and responsive

*Incomplete liveness*: update programs, but using devs actions (buttons)

Level 4: Informative, significant, responsive and live

Liveness

Steven L. Tanimoto – VIVA: a visual language for image processing – Journal of Visual Languages & Computing (1990)

# Traceability in Xtext

Case 1: runtime visualization

Generated code and generated visualization

Connect code execution with visualization

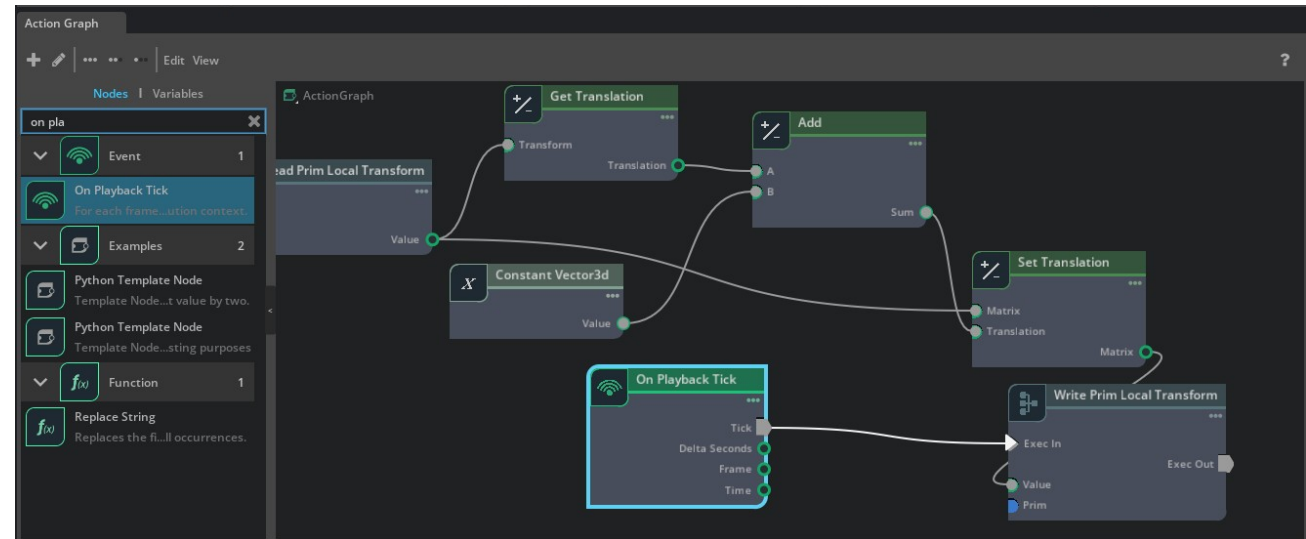
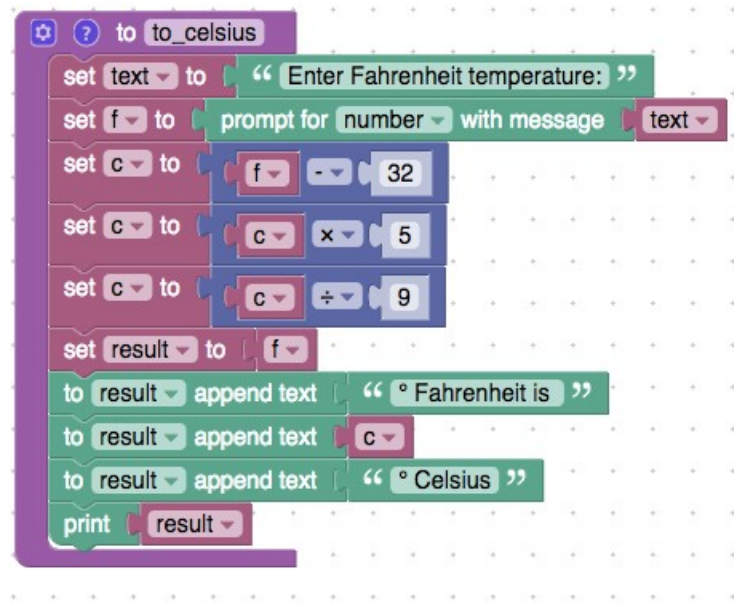
Model-aware generation

Case 2: source code

What about DSL source code?

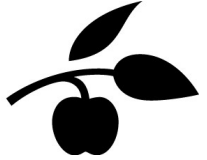


# More Examples?



# Model-Driven Software Development

## Types

**SDU** 

# Type system

If it looks like a duck...

# Types: what do you think about them?

Why do we have types?

Are they important (for you)?

# Today

(a tiny bit of) Types

# Class Description V4: Complex functions

```
system University
external legalCPR(String) : Boolean
external complexExp(Number,Number) : Number
```

```
entity Person
  attribute name is String
  attribute age is Number
  attribute cpr is String
  require age >= 0
  require legalCPR(cpr)
```

...

```
require 0 <= complexExp(10,attr)
```

S ::= 'system' ID X\* E\* R\*

X ::= 'external' ID '('(Type (',' Type)+)?')' : T

E ::= 'entity' ID (At|Q)\*

At ::= ('attribute' | '-' ) ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID<sub>E</sub> ('is' | '->') ID<sub>E</sub>

C ::= ID<sub>E</sub> ('1-1' | '\*-\*') ID<sub>E</sub>

Exp ::= ... | Xc

Xc ::= ID<sup>x</sup> '('(Exp (, Exp)+)?')'

# Class Description V5: Type checking

```
system University
external legalCPR(String) : Boolean
external complexExp(Number,Number) : Number
```

```
entity Person
  attribute name is String
  attribute age is Number
  attribute cpr is String
  require age >= 0
  require legalCPR(cpr)
require legalCPR(age)
require age >= name
  ...
```

S ::= 'system' ID X\* E\* R\*

X ::= 'external' ID '('(Type (',' Type)+)?')' : T

E ::= 'entity' ID (At|Q)\*

At ::= ('attribute' | '-' ) ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID<sub>E</sub> ('is' | '->') ID<sub>E</sub>

C ::= ID<sub>E</sub> ('1-1' | '\*-\*') ID<sub>E</sub>

Exp ::= ... | Xc

Xc ::= ID<sup>x</sup> '('(Exp (, Exp)+)?')'

# Types

Very complex topic with a lot of algebra

It is still important for us to know how this works in the context of programming languages

To prevent *execution errors*

Non-trivial property!



# Types: Execution Errors

## Trapped errors

Computation stop immediately

e.g., illegal instruction, illegal memory reference, ...

## Untrapped errors

Go unnoticed and may cause arbitrary behavior

e.g., data corruption, improperly accessing a legal address

# Types: typed and untyped languages

Typed languages

- Variables can be given types

Untyped languages

- Range of variables is not restricted

- No type (or single universal type)

# Types: Explicit / Implicit

Typed languages

variables can be given types

Explicitly typed: types are part of the syntax

Implicitly typed: types are not part of the syntax

# Types: Type system

Keep track of the types of variables and expressions

Determine whether programs are well-behaved

# Types: Well-behaved programs

Keep track of the types of variables and expressions

Determine whether programs are **well-behaved**

Does not cause *forbidden errors*

Untrapped errors

A program is **safe** if does not have untrapped errors

Some trapped errors

# Types: Well-behaved programs

Check good behavior: typechecking

Compile time check: statically checked

Run time check: dynamic checked

Typechecker: algorithm that performs typechecking

Strongly checked language: all programs are well-behaved

No forbidden errors (in particular, safety)

Some errors can occur (programmer's responsibility)

# Types: Well-behaved programs

Check good behavior: typechecking

*Compile time check: statically checked*

Runtime check: dynamic checked

Typechecker: algorithm that performs typechecking

Why do we check both on compile and runtime?

Can we only check at compilation time?

# Types: Well-behaved programs

Check good behavior: typechecking

*Compile time check: statically checked*

Runtime check: dynamic checked

Typechecker: algorithm that performs typechecking

Why do we check both on compile and runtime?

Can we only check at compilation time?

e.g., array bound, division operations ( $1 / 0$ )

Other examples?



# Types: Safety

Typed and Safe: Java

Untyped and Safe: LISP

Typed and Unsafe: C

Untyped and Unsafe: Assembler

# Types: Safety

Why unsafe?

Execution time

Why not unsafe?

Development and maintenance time

Security holes

# Types

[Very] advanced topics

Polymorphism

Subtyping

Type abstraction

Type inference

...

Type in Xtext

# Types in Xtext

Explicit type:

- Add types to grammar

- Verify correct use of variables / functions / etc. using validation checks

Define the types of your DSL

Define types for constant expressions: numbers, strings, bool

Define a type for expressions: addition, comparison, function call, var ref

For Math Exp, check type and expected type of sub statements

- Addition:  $a + b$ ,  $a$  and  $b$  should be numbers (*or something else?*)

# Types in Xtext: in-class exercise

Explicit type:

*Add types to grammar: upload content of xtext file*

Verify correct use of variables / functions / etc. using validation checks

Define the types of your DSL

Define types for constant expressions: numbers, strings, bool

Define a type for expressions: addition, comparison, function call, var ref

For Math Exp, check type and expected type of sub statements

Addition:  $a + b$ , a and b should be numbers *(or something else?)*

# Types in Xtext

Explicit type:

- Add types to grammar

- Verify correct use of variables / functions / etc. using validation checks

Implicit type:

- If interpreter: model-base execution should check (assignment 1)

- If generate code: Do nothing! (use the base programming language)

- Or if needed, generate also the checkers yourself

- `If( b == 0 ) throws Error('Divided by 0') else a/b`

# Validate types and functions: extension

- Validate types of expressions

  - An addition can only add numbers

- Validate return type of function calls and expressions

  - The function should return the right type to work in the right context

  - An expression also has a return type that should be checked

- Validate arguments of functions

  - Right amount of arguments

  - Right types of arguments



# Summary

# Summary

## Type systems

```
system University
external legalCPR(String) : Boolean
external complexExp(Number,Number) : Number
```

```
entity Person
  attribute name is String
  attribute age is Number
  attribute cpr is String
  require age >= 0
  require legalCPR(cpr)
require legalCPR(age)
require age >= name
  ...
```

